Development Environments:

Version Control Systems. Git.

Raúl Palao



Software Development Evolution

Software development evolved from solo programming to global collaboration.

PAST: Solo Programming Era

- **Only developers**
  Individual engineers working in isolation without team synchronization tools.
- **Short software lifespans**
  Applications built for immediate, bounded use cases with minimal long-term support.
- **Limited files**
  Small, simple codebases consisting of localized file structures and assets.

PRESENT: Global Collaboration Era

- **Exponential growth**
  Massive expansion in codebase scale, ecosystem complexity, and feature depth.
- **Collaborative global teams**
  Distributed engineering teams working asynchronously across time zones.
- **Long product lifecycles**
  Decade-long systems requiring strict maintenance, CI/CD, and robust governance.

CASE tools

CASE tools are software that helps create software.

KEY ACRONYM

CASE

DEFINITION

Computer-Aided Software Engineering

PRIMARY PURPOSE

Assists developers throughout the entire software lifecycle.

CORE GOAL

Protect expensive software investments and streamline complex development phases.

CASE tools support every phase of development.

Coding

- IDEs
- Code editors
- Compilers `(javac)`
- Version Control Systems `(Git)`

Testing / Debugging

- Debuggers
- Profilers

Management

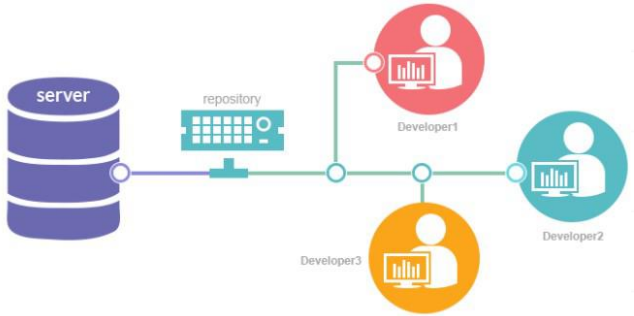
- Issue trackers `(Mantis)`
- Project managers

Version Control System (VCS)

Version Control Systems

VCS tracks file evolution during integration and maintenance.

Function	**Manages file additions and modifications over time.**
Necessity	**Crucial for collaborative network environments to prevent overwritten code.**



VCS prevents overwritten code and unresolved conflicts.

- Revert to previous versions after errors.
- Recover lost code.
- Track who, what, when, and where changes occurred.
- Experiment safely without breaking the main application.

VCS Architectures

Traditional Architecture

Centralized

SVN(Subversion)

- Open source, older standard architecture.
- Relies on a single central server for repository history and file versioning operations.

Modern Paradigm

Distributed

Git

- Open source, created by Linus Torvalds.
- Gives every developer a full local clone of the entire repository history.

Git is the overwhelming modern standard for version control systems.

Centralized VCS relies on a single server repository.

Architecture

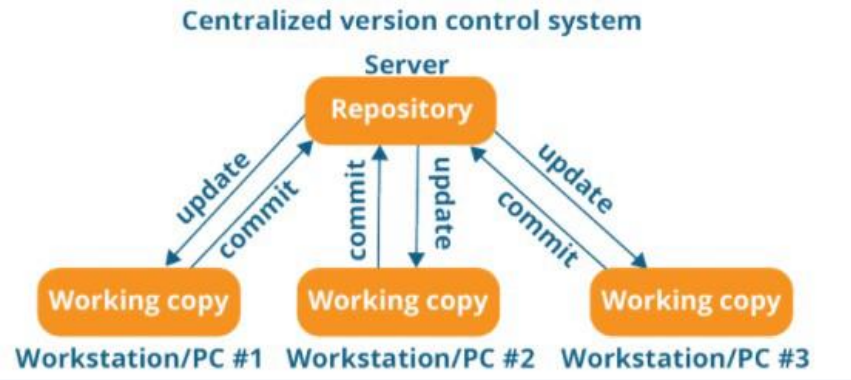
One central server holds the repository.

Workflow

Clients connect directly to the central server to commit or update.

Example

Subversion (SVN), Concurrent Versions System (CVS).



Distributed VCS relies on various full repository copies.

Architecture

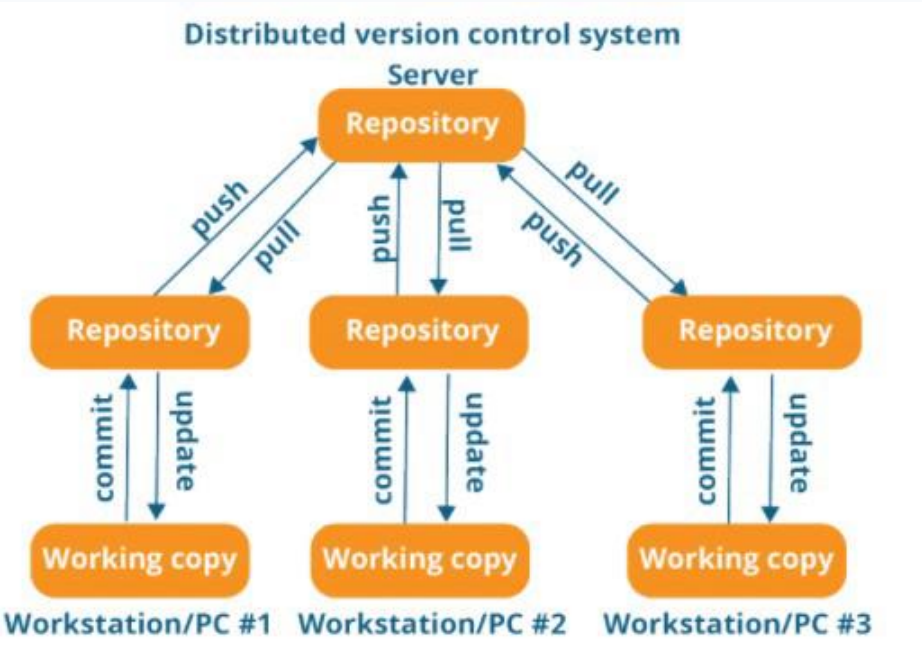
Every client has a complete local repository with full history and version tracking.

Workflow

`Push` **(send changes)**
Pull **(extract changes)**
`Clone` **(copy full repository)**

Example

Git, Mercurial.



GIT



Master the core vocabulary.

01 / STORAGE

Repository

Database storing files.

02 / LOCAL WORKSPACE

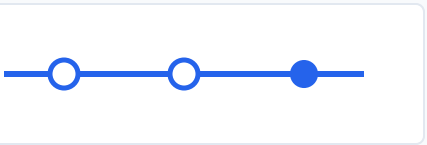
Working Copy

Local directory for edits.

03 / PRIMARY LINE

Trunk / Main

Primary code line.



04 / ISOLATED WORK

Branch

Private copy for isolated work.



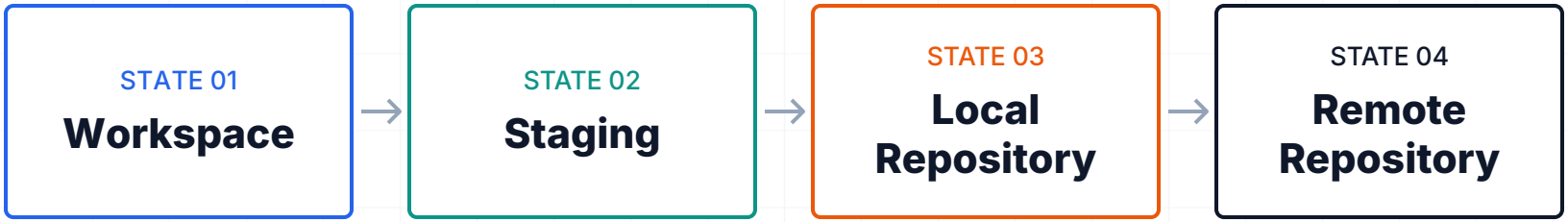
05 / COLLISION

Conflict

Contradictory changes applied simultaneously.



Git manages local files through distinct states.



CORE PRINCIPLE: SAFETY

Protects code locally before sharing it globally (eg, GitHub).

Standard operations manage the flow of code.

Core version control actions for tracking and integrating changes.

ADD

Track a file for the first time.

COMMIT

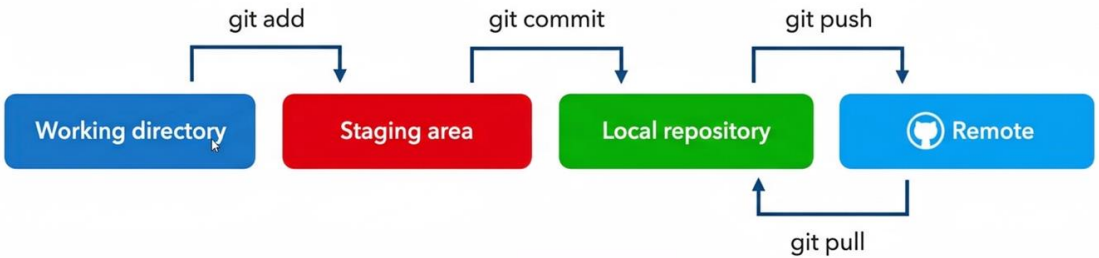
Save modified filesto the repository.

PUSH

Upload the modifications to therepositoryserver.

PULL

Get the latest repository version.



GIT installation and GitHub registration



Git Installation: Linux, Windows, and macOS Setup

Installation varies slightly by OS:

Package Managers (Linux)

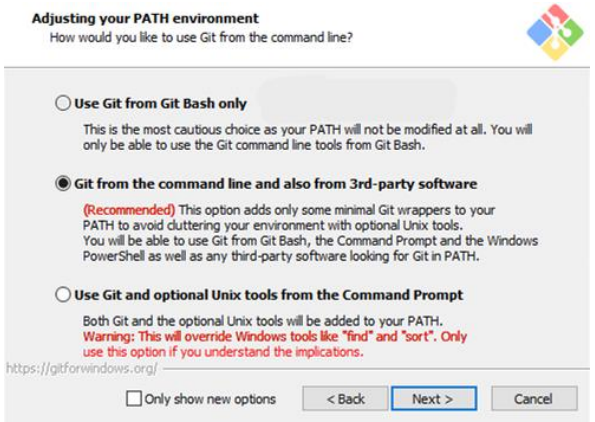
Ubuntu / Debian

```
>_ $ sudo apt-get install git-core
```

Graphical Installers

Windows & macOS

Download the visual installer from `git-scm.com`.



GitHub Registration: Account Setup Process

Create and configure your remote repository account in a few simple steps.

01

Sign Up

Go to `github.com/signup` and complete the account creation form.

02

Email Verification

Verify your email address using the confirmation link sent to your inbox.

03

Security Configuration

Set up two-factor authentication (2FA) and generate a `Personal Access Token (classic)` for command-line access.

GitHub Authentication: Personal Access Tokens

Securely authenticate with GitHub from the command line.

Security Requirement

GitHub now uses tokens instead of passwords for CLI access.

01

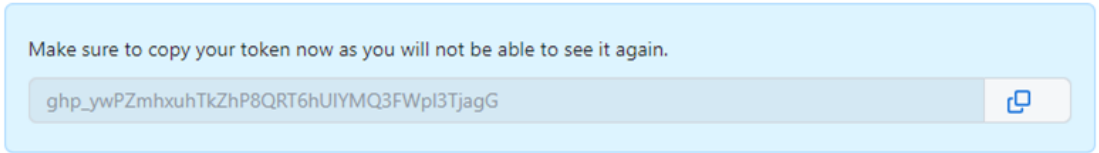
Navigation Process

`Settings>Developer settings>Personal access tokens`

02

Action Required

Generate a new token (classic) and save it securely (this will be your password).



Practical Example I - Creating a project from scratch



Working Environment: VS Code Integrated Terminal

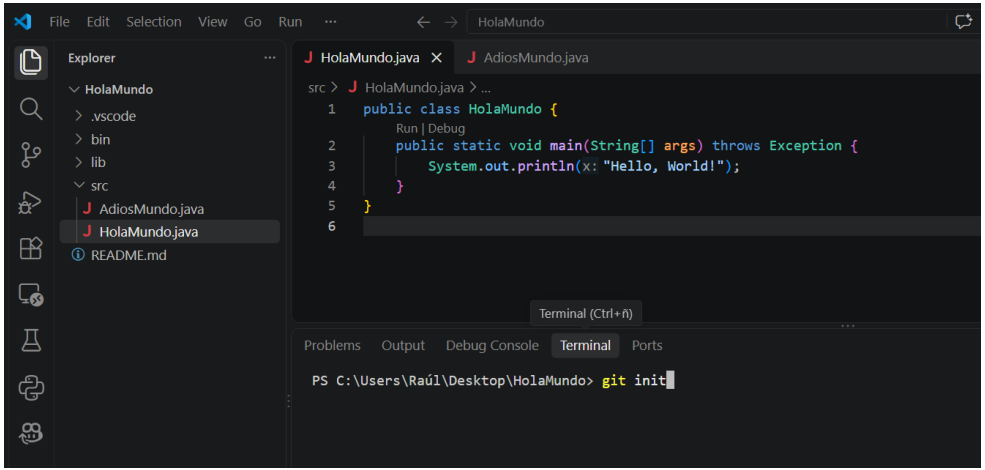
We will use the built-in console to run all Git commands.

1. Quick Opening

Use the keyboard shortcut `Ctrl + Shift + ñ` to open the terminal instantly.

2. Everything in one place

Avoid switching between external windows. You can work with code and run Git Bash or PowerShell commands in the same interface.



Creating an Empty Repository on GitHub

Step-by-step guide to starting your project in the cloud

GitHub

01

Access and Start

Log in to your GitHub account (`github.com`) and click on the button"New".

02

Repository Configuration

Give your repository a name (e.g. `01_REP_INITIAL`. Optionally, add a short description. Select whether it will be Public **either** Private**. In this case we will select** Public**.**

03

Empty Creation (Important)

Unmark **the boxes of *Add a README file*, *.gitignore* and *License* to avoid conflicts when linking your local code.**

04

Finish and Get URL

Click on "Create repository". Copy the generated URL to link it from your terminal.

Git Flow Setup and Startup

Command guide for setting up a repository

⚠ Important: **The first 4 commands are only executed only once when configuring the project.**

Initial Setup

ONLY THE FIRST TIME

1. User identification

```
$ git config --global user.name "Your Name"
```

2. Associated email address

```
$ git config --global user.email "your@email.com"
```

3. Initialize local repository

```
$ git init
```

4. Link to remote repository

```
$ git remote add origin <URL>
```

First Commit and Submission

REGULAR FLOW

5. Add files to the staging area

```
$ git add .
```

Prepare all changes for commit

6. Save changes locally

```
$ git commit –m "Initial commit"
```

Create a checkpoint with a message

7. Upload repository to the cloud

```
$ git push origin master
```

Send the saved changes to the remote

View of the Repository on GitHub

Web interface for file management and history tracking

1. File Explorer

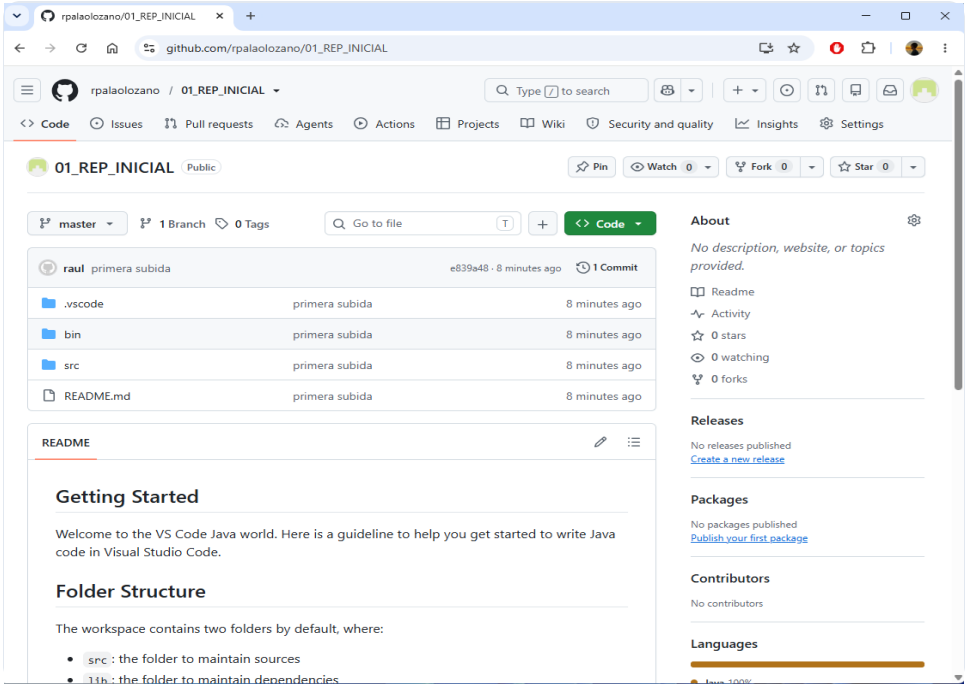
It allows you to navigate through the project structure in the cloud and view the content directly on the web.

2. Commit History

It displays the complete change log, descriptive messages, and authors for each version.

3. Remote Link (URL)

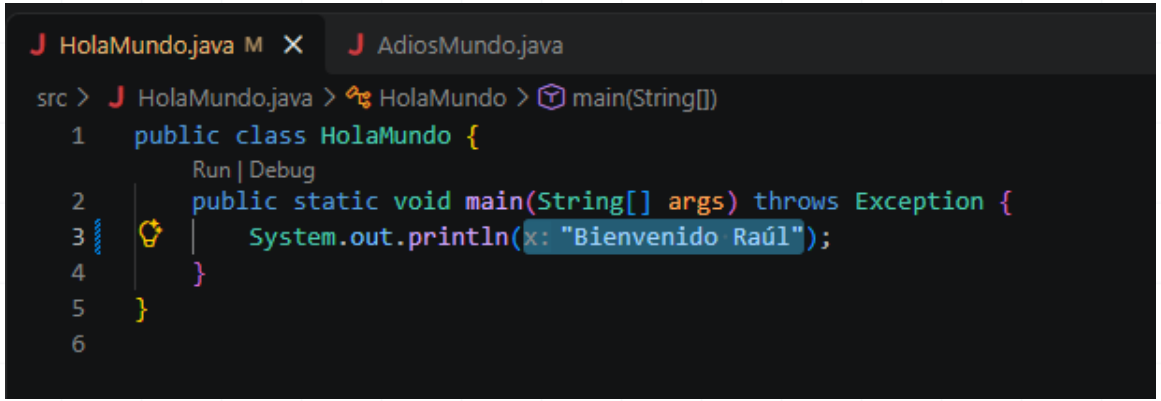
Address used for synchronization via commands `git push` and `git pull`.



Workflow When Modifying Code in VS Code

Steps required to synchronize your local changes with the remote repository

> ⚠ **Step zero is essential! Always do `git pull` before starting work.**
> **Download the latest changes from the remote server to avoid merge conflicts when uploading your own changes.**



bash — git workflow

```
$ git pull origin master

$ git add .

$ git commit -m "msj"

$ git push origin master
```

Practical Example II - Cloning



Cloning an Existing Repository (`git clone`)

Direct download of a remote project configured and ready to work



💡 **Flow simplification: No requirement `git init` neither `git remote add origin`.**
The command creates the local folder, initializes Git, and automatically links the remote URL.

1. Automatic Copy and Configuration

`git clone <URL>`. **Download the entire history and configure the remote `origin` implicitly.**

2. Ready for theFluxuryrregular

Once cloned, you enter the created folder and can start editing directly, making `git add` and `git commit`.

3. Direct Synchronization

You can use directly `git pull` and `git push` without additional configurations.

bash — clone and start work

```
# 1. Clone the remote repository
$git clone URL


#2. Work directly (Add + Commit)
$git add .

$git commit -m "My changes"


#3Send to remote
$git push origin master
```

History Inquiry (`git log`)

Viewing change logs in the local console and on GitHub

1. History in Local Console

bash – git log

```
$git log
commit 38d3d72...(HEAD -> master, origin/master)
Author: Sergio Badal < sergio@email.com >
Date: Thu Oct 29 14:19:39 2020 +0100
  my third commit
commit f592926...
Author: enyzing < enyzing@email.com >
  Update holagit.txt
commit e5f5222...
Author: Sergio Badal < sergio@email.com >
  my first commit
```

💡 Note:The HEAD pointer always points to the last commit made.

2. History on the GitHub Web

